

DEEP DIVE

The Constraint Moved

Nine in ten professional developers now use a coding agent every week. Nineteen studies on what that produced, and sixteen named teams and practitioners on what they do differently because of it — how work gets sliced, what goes in the instruction file, and what review turned into.

21 August 2026 | Issue 23 | Engineering Digest | Deep Dive

15 – 21 AUGUST 2026

16

TEAMS &
PRACTITIONERS

19

STUDIES &
DATASETS

5

ROLLOUTS
TRACED

1

NAMED BANK
ROLLOUT

90%

Nine in ten professional developers use a coding agent at work at least weekly; 68% use one daily. JetBrains, more than 15,000 professional developers, fielded May–July 2026, published August 2026.

FOREWORD

The largest fresh sample yet taken of coding-agent use was published this month: nine in ten professional developers now reach for a coding agent at least weekly. Adoption is settled. What teams do with it is not, and 2026 has been the year engineering organisations started writing down what they actually changed — how work gets sliced, what goes in the file the agent reads first, how several agents running at once are handled, and what happened to review.

This issue sets that practitioner record against the year's outcome evidence. Sixteen named teams and practitioners describe what they do differently; nineteen studies, surveys and

datasets describe what it produced. The two halves disagree in useful places, and the disagreements are where a team mid-rollout will find something worth comparing notes on.

HOW TO READ THIS

Three Paths Through This Issue

2 min

THE HEADLINES

The cover figure, the Executive Summary, and the verdict box in Lessons & Implications. The whole argument in miniature.

17 min

THE FULL PICTURE

Add Source 2 for the practices named teams hold, Source 3 for felt speed against measured speed, and the Counterpoint for how the two readings reconcile.

44 min

THE DEEP READ

All five source sections — the adoption ladder, day-to-day practice, the outcome evidence, the code and the people, and five rollouts side by side — then the Monzo case study, Lessons, and the rest of the week.

1 OF 9

EXECUTIVE SUMMARY

Adoption is settled. Practice is where the year's work went. (2 min)

Every measure of *input* is climbing steeply: weekly agent use, daily agent use, the share of merged code that is AI-authored, pull-request volume, pull-request size, and spend. Every measure of *outcome* that has been independently observed is flat, falling, or unmeasured: developer experience, change confidence, the security pass rate of generated code, junior skill formation, and the share of organisations running agentic AI in production rather than in a pilot. The constraint on delivery has moved from writing code to verifying, reviewing and maintaining it, and the practices that separate teams from one another now sit downstream of the keyboard.

A measurement problem has to be named before any of the numbers can be read. The word “agent” covers at least three distinct things across these sources, from a chat assistant answering a question to software with standing authority to act in production. Source 1 pins the three bands down and tags every survey with the one it measures, which is also the quickest way for a team to locate its own use on the ladder.

Source 2 is the practitioner record: how work is sliced and planned, what goes into the file the agent reads first, how teams handle several agents at once, what review became, what happened to tests, and where teams still refuse to hand anything over. Source 3 sets the modelled economics of a rollout against what telemetry and self-report recorded over the past year, including the gap between how fast the work felt and how fast it was. Source 4 examines what the output does to a codebase over four years of commit history and what assisted work does to people learning the craft. Source 5 puts five published rollouts side by side in the order each organisation took its decisions. The Counterpoint reconciles the surveys that disagree; the case study is Monzo's account of its own agent platform and the bottleneck it found on the other side.

2

OF 9

SOURCE 1 — ADOPTION

Ninety per cent is real, and it counts five different things (4 min)

THE ADOPTION LADDER

95%

Using AI tools at least weekly

90%

Using a coding agent at least weekly

55%

Regular agent use

28%

Autonomous AI as the primary way of working

11%

Agentic AI running in production

Those five figures span a factor of eight, and each of them is accurately reported by its publisher. They differ because they count different activities in different populations. The 95% and the 11% sit at opposite ends of a spectrum that the industry has not agreed a vocabulary for, and every argument about whether AI is working in engineering organisations runs aground on that spectrum somewhere. The ladder is also the quickest way for a team to locate its own position: a squad where everyone has an agent open daily and a squad merging agent-authored work unsupervised are four rungs apart, and only one of them is described by the headline number.

Three bands are enough to separate them. At the **assistant** end, a human writes and the AI suggests

JARGON WATCH

Coding agent, agent-mode, autonomous agent

A **coding agent** plans and executes multi-step work from an instruction, rather than completing the next line the way an autocomplete tool does.

Agent-mode describes the arrangement most organisations run today: the agent carries out the task, and a human still initiates it, reviews the result and merges it.

An **autonomous** or **production agent** acts with action authority and no per-step human approval.

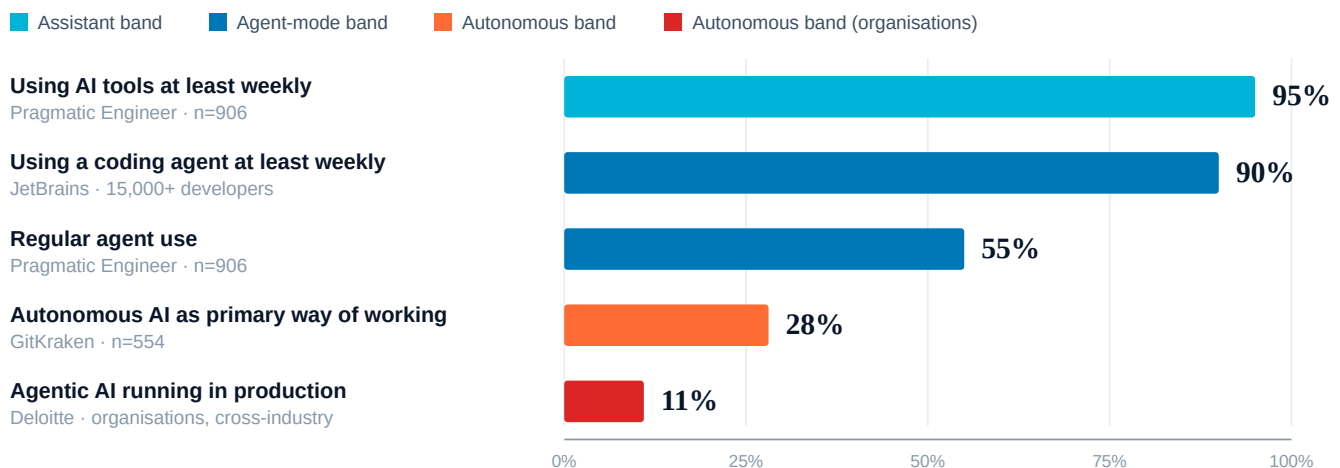
A **remote** or **cloud agent** runs on the vendor's infrastructure rather than on the developer's own machine.

or answers — autocomplete, chat, inline completion.

In the middle sits **agent-mode**, where the agent plans and executes multi-step work while a human initiates, reviews and merges it. At the far end are **autonomous** or **production agents**, acting with authority and without per-step approval. Deloitte publishes the cleanest version of that ladder in the pool, describing augmentation as “today’s reality where agents enhance human worker capabilities”, automation as “an emerging capability where agents automate tasks within processes defined by humans”, and true autonomy as “a future state in which artificial general intelligence enables agents to work with minimal oversight”. Its own numbers land where that framing predicts: 11% of organisations have agentic AI in production, 14% consider themselves ready to deploy, 38% are piloting and 30% are exploring, while 42% are still developing a roadmap and 35% have no formal agentic strategy at all. Deloitte surveys organisations across industries rather than developers, and does not disclose a sample size, so it belongs on this ladder as a measure of organisations rather than of engineers.

What the adoption numbers are counting

Five headline figures, five different definitions of “using AI”



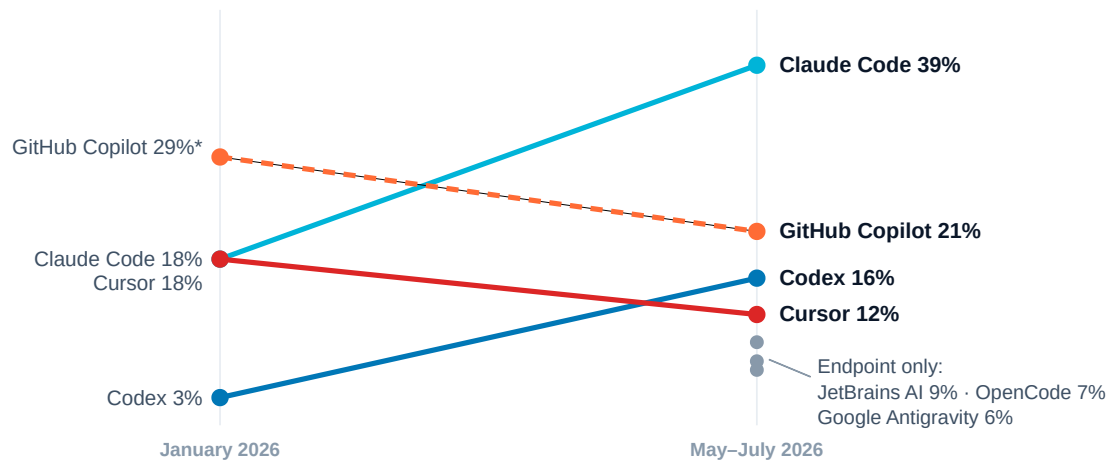
The four surveys measure different populations as well as different activities: three count developers, Deloitte counts organisations.

The three surveys, and what each one asked

JetBrains sits in the middle band. Its questions cover agents and remote or cloud agents rather than autocomplete, and its sample is more than fifteen thousand professional developers fielded between May and July 2026. Alongside the headline weekly and daily figures homed on the cover, it produced the sharpest picture yet of which tools those developers actually use. Claude Code reached 39% worldwide, up from 18% in January 2026, rising to 47% in the United States, and it is the most-used tool for 31% of developers. OpenAI’s coding agent Codex reached 16%, up from 3% in January, with awareness of it climbing from 27% to 65%. GitHub Copilot is at 21%, down from 29% a year earlier, on 79% awareness. Cursor is at 12%, down from 18% in January, and down from 28% to 16% in China. JetBrains AI is at 9%, the open-source coding agent OpenCode at 7%, and Google’s Antigravity at 6%, rising to 15% in India.

Eight months of tool-share churn

Share of professional developers using each coding agent



Source: JetBrains, AI coding agent adoption survey, more than 15,000 professional developers, fielded May–July 2026.

Four of the seven tools moved by ten points or more inside eight months. Any standardisation commitment made in January was made against a market that no longer exists in the same shape.

That churn is the practical finding of the section. Tool shares moved by tens of points in under a year, in both directions, and a single vendor doubled its share while another lost a third of its own. Standardising on one agent is a shorter-horizon bet than it looked at the start of the year, and any multi-year commitment made on current shares is being made against a market that has been reshuffling roughly every two quarters.

The Pragmatic Engineer survey draws the cleanest line between the bands of any source here, because it asked both questions separately. Across 906 respondents fielded between 27 January and 17 February 2026, 95% were using AI tools at least weekly, 75% for half or more of their work, and 56% for 70% or more of it. Regular agent use — the middle band — ran at 55%, rising to 63.5% among staff-plus engineers. Its company-size split is worth holding: Claude Code was at 75% in organisations under ten people, while GitHub Copilot was at 56% in organisations over ten thousand. Agent users were also nearly twice as likely to describe themselves as excited, at 61% against 36%.

GitKraken measures the far end. Its survey of 554 developers and engineering leaders found 96.4% of engineering organisations using AI coding tools and 84% reporting higher productivity, and it recorded the share naming autonomous AI as their “primary way of working” rising from 7.6% to 28% in nine months. That is the most striking movement in the pool and the one that most deserves its caveat. GitKraken sells the review-and-merge tooling that its own report identifies as the new bottleneck, so the finding runs in the direction of its commercial interest, and the sample is 554.

With the bands pinned down, the question that follows is what teams at the agent-mode rung actually do differently.

Sources: [JetBrains](#) (August 2026) · [The Pragmatic Engineer](#) (fielded January–February 2026) · [GitKraken](#) (14 August 2026) · [Deloitte](#) (2026)

How teams actually work with agents (9 min)

Six practices recur across the first-hand accounts published this year: how work is sliced and planned, what goes into the file the agent reads before it starts, how teams handle several agents running at once, what review turned into, what happened to tests, and where teams still refuse to hand anything over. Every one of them is a convention a team agrees and then holds.

Slicing the work, and what happened to planning

Fiona Fung, Director of Engineering for Claude Code and Claude Cowork at Anthropic, published the clearest first-hand account of what agent-assisted work does to planning ceremonies. She wrote in June: “When I first joined the Claude Code team, we wrote a pretty good six month roadmap, and then because of Claude Code, so many things changed that it was out of date by month three.” What replaced it has a name: “I call it just-in-time (JIT) planning, almost like JIT compiling: how do you do just the right amount at the right time? Our planning ritual shifted away from design docs toward discussions in PRs or prototypes.” Product review went the same way — rather than reviewing a proposal, the team builds it, puts internal users on it and acts on what comes back. Fung also names the governance mechanism that makes the rest possible: team members have explicit permission to question and kill processes that have stopped making sense.

Mitchell Hashimoto, the creator of Vagrant and Terraform now building the Ghostty terminal, describes a scheduling rule rather than a productivity aspiration. “If I’m coding, I want an agent planning. If they’re coding, I want to be reviewing.” The human and the agent are never in the same mode at the same time, which turns the aspiration of always having an agent working into something a calendar can hold. The corollary is that slow work gets queued at transitions — before a meeting, before the end of the day — and the canonical candidates are research, library comparison and edge-case enumeration, because none of them needs real-time attention.

Which work goes first is a decision several teams have made the same way. At Uber, Engineering Director Anshu Chada told a Pragmatic Summit session in March that the deliberate first target was toil: “What we found is when we push some of the boring stuff to AI — upgrades, migrations, bug fixes — not only does it result in much higher satisfaction from our engineers, but they’re able to push our product and create features for end users in ways that we didn’t even think were possible.” Hashimoto has a matching move for the team’s sceptics, and it starts nowhere near code generation: “There’s a lot of people like, ‘I don’t want it to write code for me.’ But just delegate some of the research part.”

The instruction file as a team artefact

Cloudflare's developer-productivity team found that most agent failures traced back to one thing. "Usually the problem was local context: the model didn't know the right test command, the team's current conventions, or which parts of the codebase were off-limits. That pushed us toward AGENTS.md: a short, structured file in each repo that tells coding agents how the codebase actually works and forces teams to make that context explicit." AGENTS.md is a plain-text file kept at the root of a repository that coding agents read before they begin work. Cloudflare's framing of it is worth holding: the file exists to force teams to write down context they otherwise keep in their heads. Files stay short and high-signal because they occupy space in the agent's context window, and they were generated across the estate rather than left to each team to start from a blank page.

Birgitta Böckeler, who leads AI-assisted delivery at Thoughtworks, gives the whole category a name and therefore an owner. "The term harness has emerged as a shorthand to mean everything in an AI agent except the model itself - Agent = Model + Harness." The harness covers instruction files, bootstrap scripts, linters, type checkers, test suites, architecture checks and review agents, and Böckeler splits it in two. *Guides* steer the agent before it acts: instruction files, architecture notes, how-to docs. *Sensors* tell it whether what it did was right: linters, types, tests, structural checks. Her observation from client work is that most teams over-invest in guides and under-invest in sensors, and that a sensor firing only in CI fires too late to close the agent's own correction loop. The useful ones fire while the agent is still working.

Simon Willison makes the same point from the other end, about the repository itself. "There's a lot to be said for keeping your codebase high quality because the agent will then add to it in a high quality way. And honestly, it's exactly the same with human development teams—if you're the first person to use Redis at your company, you have to do it perfectly because the next person will copy and paste what you did." The practical form of that is a golden-path repository template — a readme, CI and one small test already in the house style — rather than a document asking people to be careful.

Running several at once

Parallel agent use arrives on its own, and the honest mechanism is dead time. Ty Smith, a principal engineer at Uber, described it plainly: "as an engineer, you're giving a prompt and waiting for something. While it's running and you're waiting you're thinking: 'What am I going to do? Have a coffee or browse Reddit? Might as well kick off another background agent.'"

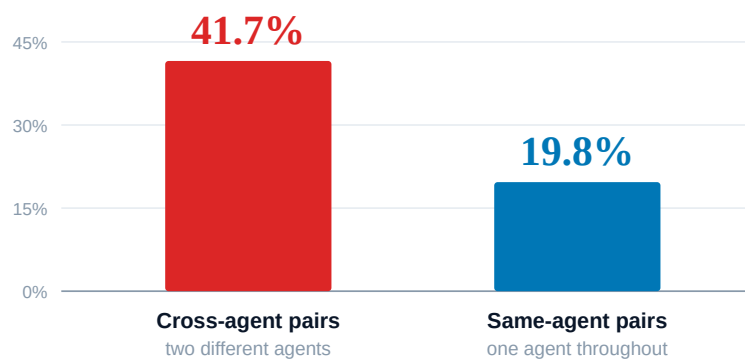
Hashimoto runs a more deliberate version where confidence is low: the same task against several models in separate checkouts, then adjudication between the outputs. It is a wasteful pattern that is only rational because machine time is cheap and review time is not.

The volume is already substantial: GitHub's Octoverse recorded more than a million coding-agent pull requests created between May and September 2025. The evidence for why parallel

work needs a team norm rather than an individual habit comes from a study of 33,596 agent-authored pull requests.

When two agents work the same repository

Conflict rate across 747 replayed three-way merges of co-active pull-request pairs



THE SAMPLE

33,596

agent-authored pull requests

40.2%

of repositories carry co-active agent PR pairs

79.4%

of all agent PRs sit in such a pair (95.0% at a one-week window)

Cross-agent overlap is currently rare: only 0.5% of concurrent pairs involve different agents, in about 4.3% of repositories.

Source: Xu, Subramanian & Karthik, arXiv, submitted 6 July 2026 · 2,807 repositories.

The honest reading of this chart is forward-looking. Cross-agent overlap happens in only 0.5% of concurrent pairs today, across roughly 4.3% of repositories, so the 41.7% figure describes what happens when two agents meet rather than what happens in the median repository. It is the evidence-based argument for keeping one agent per codebase before multi-agent overlap becomes normal.

A second analysis, of 25,264 agent-generated pull requests across 2,361 repositories, reported by Chris Stokel-Walker for LeadDev in July, describes what happens to the humans around that work: “in 79% of agentic PRs, the same developer both reviewed and modified the AI’s contribution. Just one in eight workflows involved multiple humans.” Most teams believe they have a norm against unreviewed agent merges. That is the measurable form of the norm quietly breaking. Courtney Miller, an assistant professor of computer science at George Washington University, put the underlying problem in one line: “Software development is not an isolated task, and writing new code was never the hard part”. The countermeasures named in the same piece are all team-level and cheap: a Q&A channel, a standing agent item in stand-up, demos against the team’s own systems rather than toy repositories, and hack weeks.

What review became

Addy Osmani, an engineering leader at Google, names the mechanical change that explains the review-time blowout better than volume does. He relays one developer’s line about reviewing an agent’s pull request — that it made them “the first human being to ever lay eyes on this code” — and then draws out what follows from it.

“In normal review the author already understood the change and you were checking their work. With an agent PR, nobody has reconstructed the why yet, and the reviewer is the first to try.”

— Addy Osmani, *Agentic Code Review*, 15 June 2026

Human-authored review is verification: someone already built the model of why, and the reviewer checks it. Agent-authored review is reconstruction, which is a different and more expensive job. Three countermeasures follow directly. The first is a pull-request intake standard — a statement of why the change is being made, a diff of readable size, and evidence the tests were actually run — which pushes the reconstruction work back to the submitter, where it is cheap. The second is size: “Agent PRs run large, 51% larger on average in the Faros data, and reviewer engagement is one of the strongest predictors that a PR merges at all. A large unreviewable PR gets rejected outright or, worse, rubber-stamped.” The third is risk tiering keyed to the code path rather than the author: a lint and a glance for configuration, and type checks, tests, more than one automated reviewer, a domain expert and a security look for payments and authentication paths.

Stacking review bots does not substitute for the human. Across a 146-pull-request comparison Osmani reports, “Of 617 distinct flagged locations, 93.4% were caught by exactly one of the four tools. 6% by two. Almost none by three. None at all by all four.” The tools are complementary rather than redundant, which is an argument for a second automated reviewer and against treating agreement between them as assurance. Osmani names the failure that follows when the humans step out: “A confident ‘looks good’ with no human anywhere in it is borrowed confidence: the system’s certainty becomes yours, and nobody actually understood anything.”

Figma’s security engineering team states its gate in one sentence: nothing merges without human review, a static-analysis pass and an agentic review, with failover between model providers so that an outage at one vendor cannot let a pull request through unreviewed. Two frontier models run independently and either one raising a finding escalates it — deliberate redundancy on the grounds that they miss different bugs, which is the same conclusion Osmani reached from outside data.

Tests stopped being optional

The most portable convention in the whole pack fits in a sentence, and Willison opens every agent session with it. “Every single coding session I start with an agent, I start by saying here’s how to run the test... So I say run the test and then I say use red-green TDD and give it its instruction.” Red-green refers to writing a failing test first and then making it pass; every capable coding agent already knows the term, so the convention costs a handful of words. The reason it matters has changed. The historical objection to tests was the engineer time they cost to write and maintain: “They’re free now. They’re effectively free. I think tests are no longer even remotely optional.” A test suite is now the mechanism by which an agent can tell whether it has finished, which means a team that skipped tests has no such mechanism. Willison is equally clear about the limit: a green suite does not prove the service boots, so the agent is made to exercise the running system and leave a written record of what it exercised, as an artefact the reviewer can read.

Testing at organisational scale is where adoption thins most sharply. Capgemini's World Quality Report, 2025–26 edition, puts 43% of organisations experimenting with generative AI in quality engineering and only 15% having scaled it enterprise-wide.

Where teams still say no

The next step past running agents in parallel already has a shape. Armin Ronacher, the creator of Flask and formerly a principal architect at Sentry, describes it: “The pattern is the same everywhere though: work is put into a queue of sorts, a machine picks it up, attempts it, stops, and then some harness decides whether that was actually the end.” The epigraph he opens with, from Boris Cherny, states the shift in job description: “I don’t prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops.”

Ronacher is also the clearest voice on where that stops working, from his own practice: “My current status is that I have not had much success with this way of working for code I deeply care about which turns out to be quite a lot of code.” The line he draws is about the shelf life and invariant density of the output rather than the size of the task. Loops work on transformations of code that already exists — ports, migrations — and on deliberately short-lived artefacts such as performance experiments and security sweeps. They work badly on new, long-lived code with load-bearing invariants. The failure mode is specific enough to teach a reviewer to spot: “Present-day models tend to produce code that is too defensive, too complex, too local in its reasoning. They avoid strong invariants. They add fallbacks instead of making bad states impossible. They duplicate code, invent bad abstractions, and paper over unclear design with more machinery.” Each iteration adds a small defence, and the system becomes less understandable while looking more robust.

The same boundary is drawn formally in operations. PagerDuty’s March survey of 1,000 leaders and developers across seven markets found 44% requiring human involvement when AI remediates customer-facing systems and 43% requiring it for cross-functional incident coordination. Ronacher’s personal version of the bar is a statable team norm: being able to explain what the system does under pressure without asking the model first.

These are the conventions teams are holding. Whether they show up in the outcome measures is the subject of the next section.

Sources: [Fiona Fung, Anthropic](#) (3 June 2026) · [Gergely Orosz on Mitchell Hashimoto, The Pragmatic Engineer](#) (25 February 2026) · [Gergely Orosz on Uber, The Pragmatic Engineer](#) (10 March 2026) · [Cloudflare](#) (20 April 2026) · [Birgitta Böckeler, martinowler.com](#) (2 April 2026) · [Simon Willison](#) (14 March 2026) · [Addy Osmani](#) (15 June 2026) · [Chris Stokel-Walker, LeadDev](#) (28 July 2026) · [Armin Ronacher](#) (23 June 2026) · [Figma](#) (23 July 2026) · [GitHub Octoverse](#) (28 October 2025) · [Xu, Subramanian & Karthik, arXiv](#) (6 July 2026) · [Capgemini](#) (2025–26 edition) · [PagerDuty](#) (17 March 2026)

Felt speed against measured speed (4 min)

DORA modelled what an AI rollout should be worth over its first year. DX measured what actually happened across its customer base over the same period. The distance between them is the reason a team can hold a set of practices that plainly work and still find the quarterly numbers refusing to move, and the reason the sensation of going faster is a poor instrument.

The shape DORA expects

DORA — the research programme run by Google Cloud, and not the European regulation of the same name — publishes a report on the return on investment of AI-assisted software development, version 2026.1 of which was last updated on 22 April 2026. Its central claim, drawn from a model that does not separate assistant use from agent use, is that value arrives in a J-curve: an initial dip before the return, driven by three named forces. The first is the learning curve. The second is what DORA calls the **verification tax**, “imposed by reviewing AI-generated code”. The third is pipeline adaptation — the work of adjusting downstream testing and change-approval processes to handle a larger volume of code arriving faster. DORA’s own name for the dip is “the tuition cost of transformation”. That J-curve led Issue 11 and was the first Quick Take in Issue 12; here it is a source rather than a fresh finding.

The shape DX measured

DX publishes engineering-effectiveness telemetry across its customer base, and it is a vendor writing about the metric it sells. Its Q2 2026 benchmarks, carried in its newsletter of 14 August 2026 and its Q2 report, put measured adoption at 95% — telemetry that counts every band of the spectrum together, from autocomplete to agent-mode — and found that more than half of merged code is now AI-authored. These are the aggregate numbers a

DORA’S SAMPLE MODEL

A published illustration, not a measurement. DORA models a 500-person engineering organisation at a fully loaded salary of \$176,000.

Change failure rate	5 → 6%
Negative downtime impact	\$344K
First-year return	~39%
Value modelled	~\$11.6m
Investment modelled	~\$8.4m
Payback period	~8 months

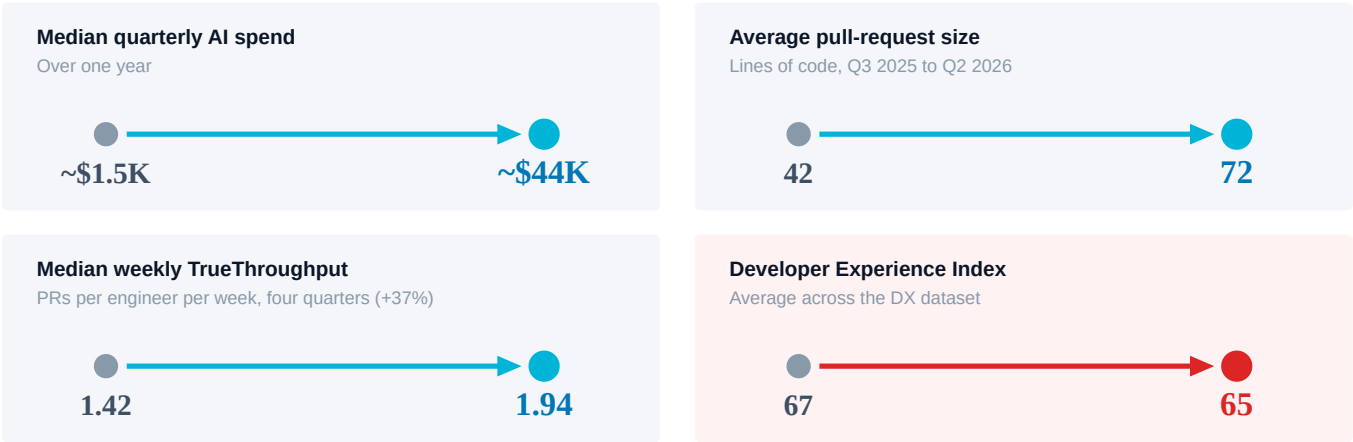
Change failure rate — the share of deployments that cause a failure needing a fix or a rollback.

team’s own dashboard is a sample of. Average pull-request size has increased from 42 to 72 lines of code. Time savings are now over six hours a week on average. Median quarterly AI spend grew from roughly \$1.5K to roughly \$44K in a year, and AI spend has reached roughly 28 times year-ago levels among the largest companies in the dataset. Median weekly TrueThroughput — DX’s own measure of shipped work per engineer — rose 37% over four quarters, from 1.42 to 1.94 pull requests per engineer per week.

Against those four rising series sits one that fell. The average Developer Experience Index, DX’s composite measure of how effective engineers find their own working environment, slipped from 67 to 65. Underneath it, code maintainability is improving while change confidence is declining, even though those two measures have historically been closely correlated. The innovation ratio has increased by only about one percentage point.

Spend, size, throughput, experience

Four DX measures, each on its own scale and its own period — start value to end value



Each series runs on its own scale and its own period; the panels are not plotted on a shared axis. Source: DX. O2 2026 benchmarks and O2 report (newsletter dated 14 August 2026).

Three rising series and one falling one, on four different scales and four different periods. Spend, size and throughput all moved up together; the experience measure moved the other way.

The time dimension

DX’s plateau study, led by Brian Houck, adds the dimension the benchmarks cannot show. Drawing on self-reported data from more than 500 companies between May 2025 and April 2026, it found 31.4% of developers climbing from the lowest time-savings band to the highest,

and 69.7% of those who reached their peak doing so within two quarters. Then the curve turns. Some 66.1% reported lower savings after their peak, and 50.5% of those who peaked in a single quarter never returned to that level. The median engineering organisation in the study sees a 7.8% increase in pull-request throughput.

The measurement question sits underneath all of this, and METR's study is the sharpest available evidence on it. Published in July 2025, it took 16 experienced open-source developers through 246 tasks in repositories they had worked on for an average of five years. Beforehand they forecast that AI assistance would make them 24% faster. Afterwards they estimated it had made them 20% faster. Measured, they were 19% slower. Three caveats belong with that result and travel with it everywhere it is cited: the sample is 16 people, the study is a year old, and the tooling was Cursor Pro with Claude 3.5 and 3.7 Sonnet, which is two model generations behind what the JetBrains respondents are using now.

What executives can show

Atlassian's State of Teams 2026, published on 27 April 2026, surveyed 12,035 knowledge workers and 173 Fortune 1000 executives in January and February, with 25 executive interviews alongside. It covers cross-industry knowledge work rather than engineering specifically, and it is Atlassian's own research. Its central pair is unambiguous: "89% of executives say AI increases speed, but only 6% of executives are sure they have clear examples of organization-wide AI ROI." Beneath it, 85% of knowledge workers use AI at work and 29% have embedded it into daily workflows, while 55% of executives report AI widening performance gaps between teams. The top-performing 14% of teams are 5.6 times more likely to credit AI with improved planning and 9.4 times more likely to report better collaboration.

Two findings here travel directly into a team's own week. Individual time savings decay after a peak for most developers, so a rollout that measured itself once in its first quarter is holding a number that has since moved. And the sensation of speed and the measurement of speed came apart badly enough in a controlled setting that neither can stand in for the other. What the extra output is made of is the subject of the next section.

Sources: [DORA, ROI of AI-assisted Software Development v.2026.1](#) (last updated 22 April 2026), sample-model figures via [InfoQ](#) · [DX newsletter](#) (14 August 2026) and [DX Q2 report](#) · [DX, the AI efficiency plateau](#) · [METR](#) (July 2025) · [Atlassian State of Teams 2026](#) (27 April 2026)

What the extra output is made of (4 min)

"[M]odels may be almost syntactically perfect, but they're still failing on nearly half of all tasks where security is needed." That is Chris Wysopal, co-founder and chief security evangelist at

Veracode, on the finding at the centre of the company's 2026 GenAI Code Security Report. Veracode sells application security, and its report is its own research.

The code

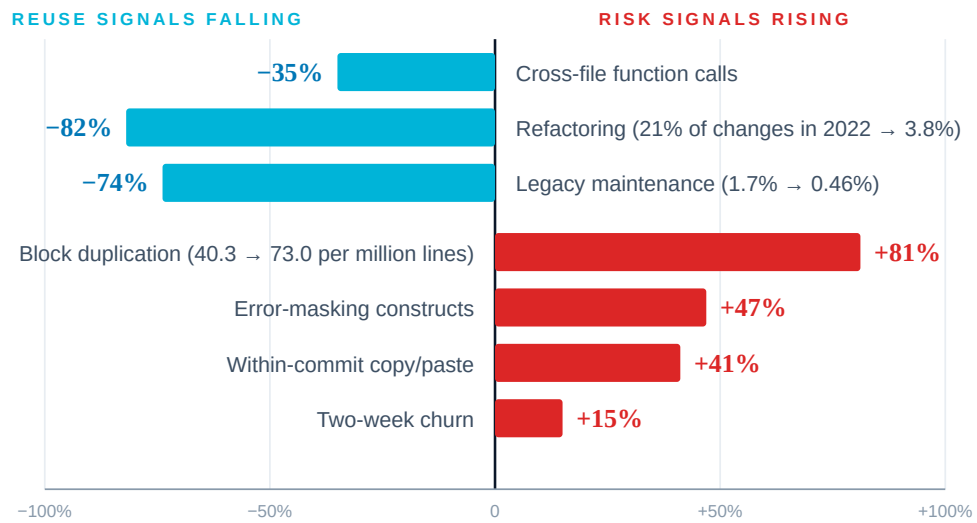
The report, released on 28 July 2026, tested more than 100 models across four testing snapshots, without security-specific prompting. The average security pass rate was 56%, meaning nearly 44% of generated code carried a known security flaw, against syntax correctness sitting near 100%. GPT-5.5 leads the field at 68%, and in Veracode's own words "six of the 11 models cluster between 50 percent and 53 percent". Reasoning models pass at 56% against 51% for non-reasoning ones. Coding-specialised models pass at 51%, marginally below general-purpose models at 52%, which undercuts the assumption that a model trained for code is safer at writing it. Large models pass at 53%, medium and small at 51% each. By language, Python passes at 63% and Java at 30%.

Capability and safety have come apart. Models write code that compiles and runs at a rate approaching certainty, and write secure code at a rate close to a coin flip, and the second figure has not moved materially in a year.

GitClear's maintainability research, published in January 2026, examines what accumulates downstream of that. GitClear sells code analytics, and this is its own dataset: 623 million changes between 2023 and 2026, measured across eight quality signals. The reuse signals are all falling. Cross-file function calls are down 35% since 2023. Refactoring — code that was moved rather than newly written — has fallen from 21% of changes in 2022 to 3.8% in 2026, a drop of 82%. Legacy maintenance has fallen from 1.7% to 0.46%, down 74%. The risk signals are all rising. Block duplication — the same run of code appearing in more than one place — is up 81%, from 40.3 to 73.0 instances per million changed lines. Within-commit copy/paste is up 41%. Error-masking constructs — code that swallows a failure instead of surfacing it — are up 47%. Two-week churn, meaning code rewritten within a fortnight of being committed, is up 15%.

The maintainability ledger

Change since 2023 across eight quality signals — 623 million changes, 2023–2026



Copy/paste is now roughly five times more common than refactoring, having been the rarer choice in 2022.
Source: GitClear, *The Maintainability Gap*, January 2026 · 623 million changes across 2023–2026.

Every signal associated with reusing existing code is falling; every signal associated with duplicating or hiding it is rising.

GitClear also compares heavy AI users against their own history. Those users out-produce non-users by four to ten times, but GitClear finds most of that gap pre-dated AI; measured against their own pre-AI baseline, the same users gained roughly 25%.

“The throughput is real, but so is the debt it accrues”

— GitClear, *The Maintainability Gap*, January 2026

The people

Anthropic published a randomised controlled trial on 29 January 2026 that measured what assisted work does to skill formation. Fifty-two mostly junior engineers, all with more than a year of weekly Python, were set to learn Trio, an asynchronous library none of them knew. The AI-assisted group averaged 50% on a mastery quiz afterwards. The hand-coding group averaged 67%. The effect size was a Cohen’s *d* of 0.738 at $p = 0.01$ — Cohen’s *d* being a standard measure of how large a difference is, independent of sample size — and the researchers describe the gap as “nearly two letter grades”. The AI group finished about two minutes faster, a difference that was not statistically significant. The largest gaps were on debugging questions.

The most useful finding sits inside the AI group. Participants who asked conceptual questions and worked through their own errors averaged above 65%. Those who delegated generation and debugging averaged under 40%. The same tool, used two ways, produced a spread wider than the gap between the groups. Anthropic sells the assistant in question, which makes this a vendor publishing a result that runs against its own commercial interest — rare enough to be worth naming.

At the level of the labour market, a Harvard working paper posted to SSRN by Hosseini Maasoum and Lichtinger, titled “Generative AI as Seniority-Biased Technological Change” — the authors’ own term for a technology that displaces work unevenly across experience levels — examines US résumé and job-posting data covering tens of millions of workers across hundreds of thousands of firms. Its direction is that after adopting generative AI, firms show declining junior employment relative to non-adopters, while senior employment trends stay largely unchanged. It is a working paper, not peer-reviewed, and its magnitudes are reported inconsistently in secondary coverage, so only the direction is carried here. It is US labour data, so any read-across to UK or Middle East hiring is inferential.

Alasdair Allan made the connected argument in his QCon London 2026 talk, “Engineering Progression When AI Ate the Middle”, reported by Ben Linders for InfoQ on 13 August 2026: the tasks AI now absorbs are the rungs junior engineers climbed, and those same juniors are the pipeline of people who will later be senior enough to supervise the AI.

Sarah Wells, a software engineering consultant formerly at the BBC and the Financial Times, names the immediate version of that problem for anyone running a team today. Asked whether juniors can evaluate what an agent hands them, she said: “I’m worried that they’re not; they can’t make the judgement about whether the agent is making a sensible suggestion”. The channel that used to close that gap is closing too. “I think code reviews are probably going to stop being the way that you share knowledge,” Wells says. “You’re going to share knowledge at a higher level than the code itself.” If that holds, the design and architecture conversations become the place knowledge transfers, and they are not currently staffed or scheduled as though they carry that load.

Both bills land on the same team, and both are paid in review attention: someone has to catch the duplicated block, and someone has to be the colleague a junior would otherwise have asked. Which organisations are paying more slowly, and in what order they built things, is the subject of the next section.

Sources: [Veracode 2026 GenAI Code Security Report](#) (28 July 2026) · [GitClear](#) (January 2026) · [Anthropic](#) (29 January 2026) · [Hosseini Maasoum & Lichtinger, SSRN](#) · [InfoQ](#) (13 August 2026) · [Chris Stokel-Walker, LeadDev](#) (28 July 2026)

6 OF 9

SOURCE 5 — ROLLOUTS

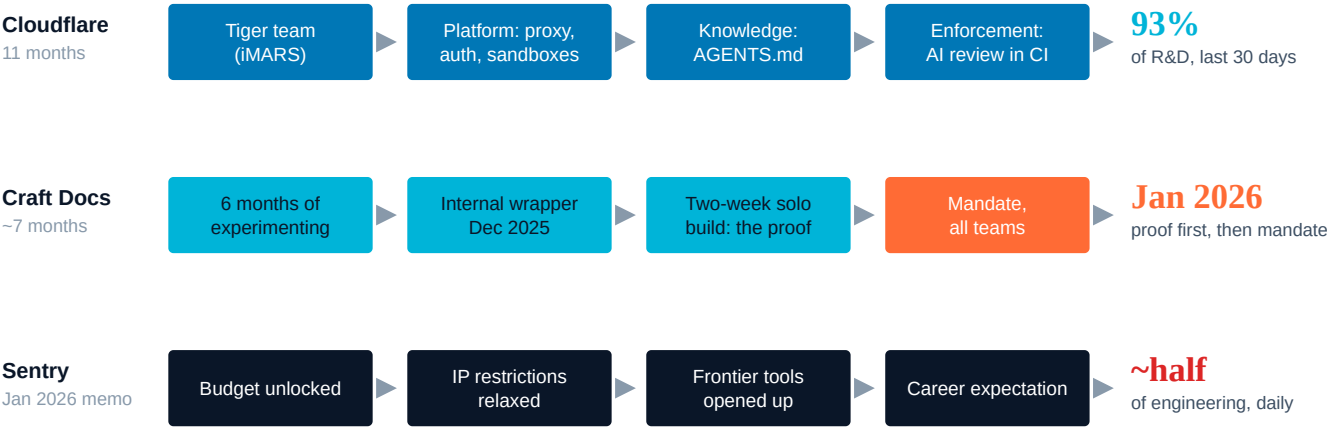
Five rollouts, and the order they did things in (9 min)

One early decision gets named by the team that made it as the one they got right. Cloudflare routed every AI request through a single proxy service from the first day, when connecting clients straight to the model gateway would have been quicker to stand up. “But centralizing through a Worker meant we could add per-user attribution, model catalog management, and

permission enforcement later without touching any client configs. Every feature described in the bootstrap section below exists because we had that single choke point.” Every rollout account published this year turns on a handful of decisions like that one, and the order they were taken in is the part a team mid-rollout can compare itself against.

Three published rollout sequences

The order each organisation took its decisions in, and the position it reports reaching



Sources: Cloudflare engineering blog, 20 April 2026 · The Pragmatic Engineer on Craft Docs, 27 January 2026 · David Cramer, Sentry, 16 January 2026.

Three orders of operations, three different positions reached. Cloudflare built the platform layer before the context layer and the enforcement layer; Craft Docs earned an internal proof before issuing a mandate; Sentry removed every access barrier it could name and still reports a plateau.

Cloudflare: choke point, then context, then enforcement

Cloudflare’s account is the most complete build order published this year, and it comes with a timeline and an owner. “Eleven months ago, we undertook a major project: to truly integrate AI into our engineering stack. We needed to build the internal MCP servers, access layer, and AI tooling necessary for agents to be useful at Cloudflare. We pulled together engineers from across the company to form a tiger team called iMARS (Internal MCP Agent/Server Rollout Squad). The sustained work landed with the Dev Productivity team, who also own much of our internal tooling including CI/CD, build systems, and automation.” MCP is the Model Context Protocol, a shared interface through which agents reach internal tools and data sources rather than each integration being wired by hand. The temporary team built it; the

WHAT STOPS TEAMS

Bessemer’s four named blockers, in its own labels.

Evaluating code quality	52%
Measuring productivity gains	46%
Managing token costs	38%
Security and IP	29%

Three of the four are measurement and verification problems. Only one is about cost.

permanent team that already owned CI/CD and build systems inherited it.

Eleven months on, “In the last 30 days, 93% of Cloudflare’s R&D organization used AI coding tools powered by infrastructure we built on our own platform.” Throughput was reported against a stated baseline rather than as a naked number: the four-week rolling average of merge requests climbed from around 5,600 a week to more than 8,700, with the week of 23 March reaching 10,952, nearly double the Q4 baseline. The team is careful about what the value came from. “None of these pieces are especially novel on their own. Plenty of companies run service catalogs, ship reviewer bots, or publish engineering standards. The difference is the wiring.”

Shopify and Ramp: plural tools, singular platform

Shopify runs the opposite of a standardisation policy on tooling, deliberately. Farhan Thawar, its VP and head of engineering, describes the exception in the company’s own terms: “At Shopify, we always have one tool for one job—except for with AI,” he explains, “since we don’t know yet which company, workflow, or model is going to win.” Engineers run several coding agents side by side. What makes that survivable is the layer underneath: a central LLM proxy — a single gateway all AI requests route through — giving “cost control, usage analytics by team and project, and the ability to switch models” without requiring a uniform tool. Human review is still required on production code, and Shopify reports reversion rates on AI-assisted code roughly equivalent to its pre-AI baselines, a figure it reports itself and nobody has audited. Thawar’s stated posture on access was blunt: “I don’t care if you have a go-to-market for this. I

don't care what the price is. How do I get this inside Shopify for every engineer?" His stated worry is the other side of it — comprehension debt, engineers gradually losing the understanding of the systems they maintain. "The brain is a muscle," he says, "If you stop going to the gym—or stop using your brain—it will atrophy."

Ramp supplies the release mechanics in the same account. Major features ship daily to an early-access tier serving "5,000+ businesses" and advance to general availability on evidence — demos, key metrics, customer feedback and a rollout plan, submitted on a template. The reviewing group commits to responding within 48 hours; if it does not, the feature ships anyway. Across the teams surveyed in the same piece, 90% were deploying AI at the core of their operations, with code generation at 92% and agentic development — Bessemer's label for agents carrying out development work rather than assisting with it — at 60%.

Sentry: every barrier removed, and still a plateau

The most useful rollout account of the year is the one that did not work as planned, published in January by Sentry co-founder and chief product officer David Cramer as an internal memo made public. "At Sentry we've been forward thinking here in the engineering org. We've unlocked budget, relaxed IP restrictions, and opened up the most cutting edge tools to our engineers. We've also been very intentional about adoption, but so far we haven't seen the organic growth we'd like in the company. About half of engineering is using AI tools in their daily workflows, but only a small subset of engineers have gotten truly comfortable with the state of the art." Three specific barriers removed —

budget, intellectual-property restrictions, tool access — and a plateau anyway. Cramer’s posture is neither a mandate nor a shrug: “this is not something that we can solve top-down, we will need folks to lean in and we are going to expect it.” He leads with his own practice, having not written code by hand in six months, and he names the cost rather than hiding it: “there will be a point where we need to address that... This is especially true within engineering where some of us are spending more than \$100/day on these tools. Yes, it’s a lot. Yes, it’s worth it. Yes, we will fix it at some point.”

Craft Docs: earn the proof, then mandate

Craft Docs, a five-year-old startup, ran the sequence in the opposite order to most. Leadership refused to commit until it had its own demonstration: “We knew we had one shot at showing users that AI is useful for them. And until we had a ‘wow’ moment ourselves, we would not go all-in on AI.” Six months of experimentation, an internal terminal wrapper built in December 2025 and a two-week solo build over the holidays came before mandatory adoption across all teams in January 2026. The budget policy is a single line — \$200 a month per person, engineer or not, with founder Balint Orosz running about \$3,000 a month on top as the heaviest user — and cost was explicitly not being optimised at that stage. The enablement ritual is the most transplantable artefact in the account: a weekly “What’s Impossible?” challenge in which teams propose something they believe would take two to three months and then attempt it in a week. One migration made the case on its own: “This migration work was estimated to take 3-4 months. It was done in a week with AI tooling.” The surprise adopters were the non-engineers, who built triage,

reporting and user-data workflows without engineering time.

Figma: the policy is the artefact

Figma's security engineers set out to point agents at their codebase and ended up writing something else. "Ninety-nine lines, 2,560 words, and 68 precedents later, this work had a side effect we did not plan for: We had written a complete threat model, in roughly the form we'd want a new hire to read on day one. The policy is the threat model." One written policy now drives three stages: review before merge, guidance during generation, and retrospective audit of a decade-old monorepo. The measurement discipline around it is unusually honest. Findings only surface to a human once a class of finding has held above a precision threshold over a rolling look-back — the mechanism that keeps reviewer fatigue from turning into rubber-stamping — and the team defines its terms plainly: "Precision is the share of reported findings that are real vulnerabilities... Recall is the share of real vulnerabilities the system catches." Those measurements run against a fixed corpus of known cases, the practice teams now call evals. The denominator is stated rather than flattered: "every task in the corpus is a bug that our historical review process, human reviewers plus static analysis (SAST), already missed. So a 75.8% union catch rate is 75.8% of the bugs that got all the way past our existing controls." Blast radius is scoped explicitly — read-only access during analysis, sandboxed execution with egress controls for fix-writing agents, and CI for agent-generated pull requests running without deploy credentials or production secrets. The agent layer is treated as production code: "Every policy change is reviewed by a security engineer and rerun against the full corpus

before it ships.” And it is affordable: “For both the models we use, a pull request review runs about \$0.50 median spend, rarely more, since most PRs have nothing to flag.”

What actually spreads a rollout

Three Microsoft researchers studied their own employer’s early-2026 rollout of command-line coding agents across tens of thousands of engineers, and their finding cuts against most enablement budgets: “first use spread primarily through social networks, retention was associated more with engineers’ coding activity than with demographics, and adopters merged roughly 24% more pull requests than they would have otherwise.” The authors attach the caveat themselves, which is the model for how any throughput figure should travel: “We use merged pull requests as our proxy for output -- acknowledging that a merged PR is not the same as the value it delivers -- and the lift persists across our four-month window.” Their recommendation is explicit — organisations should treat visible peer use as central to rollout strategy. It is a preprint and not peer-reviewed. Uber, which has the leverage to mandate if it wanted to, reached the same conclusion from practice: adoption was voluntary and peer-led by design, and slower than expected even so.

Someone has to own the scaffolding for any of this to hold. Perforce’s 2026 platform-engineering survey of 820 technology professionals found 73% of organisations with mature platform-engineering practices calling that maturity a critical or significant factor in their AI success, against 44% among the less mature — a 29-point gap — while only 28% report having a dedicated platform-engineering team at all. Cloudflare’s answer was to hand the work to the team that already owned CI/CD;

Böckeler's is that a harness is a maintained artefact with a rota rather than a project with an end date.

The bill, and the regulated case

The relaxed-spend posture has an end point, and it arrived during the first months of 2026. Chris Reed, senior director of IT finance at Priceline, described the pattern after his company began placing token limits on certain groups: "It's like the crack-cocaine epidemic... They let you try it to get you hooked on it, and now you're kind of beholden to it." The management problem is attribution rather than the total. Vitaly Gordon, chief executive of Faros AI, relayed a chief technology officer's dilemma: "One of my engineers spent \$40,000 on tokens last month, and I genuinely don't know whether I should stop him or should I go and tell everyone else to be like him." J.R. Stormont of the FinOps Foundation dates the turn: "We started hearing existential crises, and the whole conversation shifted from tokenmaxxing and 'go fast' to 'we need guardrails, how do we control this?'" Alexander Embiricos, head of enterprise at OpenAI, lists what buyers now ask for: "What visibility do you have? What auditability do you have? What token controls do you have? What is the efficiency of your models?" Those four questions are the argument for Cloudflare's day-one attribution, made in retrospect by everyone who skipped it.

The closest published analogue to a UK bank rollout is Lloyds Banking Group, which is running a four-year research partnership with the University of Glasgow that places agents directly inside software and data engineering teams and measures their performance alongside human engineers under live conditions. QA Financial, reporting the

programme in May, framed the shift in evaluation it represents: “Lloyds’ move shifts the testing question from ‘does this work in isolation?’ to ‘does it work safely in live engineering environments under regulatory constraint?’” Those are the outlet’s words rather than the bank’s; the same report paraphrases chief data and analytics officer Ranil Boteju as framing the trial around preserving auditability and explainability, and Dr Tim Storer of Glasgow as aiming to produce evidence-based guidance for large regulated organisations — an admission that no such guidance yet exists. One unexpected measurement travels with it: Richard Bishop, a lead quality engineer at Lloyds, has reported that software-testing environments produce 120% more carbon emissions than production environments, which becomes a first-order number once agents multiply test runs.

Separately, and published in June, the bank is recruiting for almost 300 agentic-AI-related roles within more than 1,000 AI roles planned for 2026, with more than 700 colleagues already shaping AI use cases, a Level 6 AI Engineering apprenticeship running a 33-apprentice cohort, and an internal academy where over 400,000 courses have been taken since January against a 67,000-strong workforce.

The enablement layer explains the spread between organisations. It does not explain why the surveys disagree with each other, which is the subject of the Counterpoint.

Sources: [Cloudflare](#) (20 April 2026) · [Bessemer Atlas on Shopify](#) (2 April 2026) · [Bessemer Atlas](#) (10 June 2026) · [David Cramer, Sentry](#) (16 January 2026) · [The Pragmatic Engineer on Craft Docs](#) (27 January 2026) · [Figma](#) (23 July 2026) · [Murphy-Hill, Butler & Savelieva, arXiv](#) (1 July 2026) · [The Pragmatic Engineer on Uber](#) (10 March 2026) · [Perforce, via InfoQ](#) (4 August 2026) · [Birgitta Böckeler](#) (2 April 2026) · [Rebecca Bellan, TechCrunch](#) (5 June 2026) · [QA Financial via Resultsense](#) (5 May 2026) · [Lloyds Banking Group](#) (22 June 2026)

Two readings of the same year (2 min)

THE ADOPTION READING

90%

JetBrains' weekly coding-agent use

28%

GitKraken's primarily autonomous working

92%

Bessemer's code-generation adoption

THE MEASURED READING

11%

Deloitte's agentic AI in production

67 → 65

DX's Developer Experience Index

56%

Veracode's security pass rate

Why the two columns reconcile

The two columns are not in contradiction. They count different bands of the spectrum, over different horizons, in different populations. Weekly tool use and production autonomy are separated by two full bands and a governance decision, and a survey of developers answers a different question from a survey of organisations. The left column is a measure of who has the tool open. The right column is a measure of what the organisation has committed to running without a human in the path.

A third difference sits underneath both. Survey answers about tool use record a state that can change in a fortnight, because installing an agent is a decision one developer can make on a Tuesday. Measures of production autonomy, security posture and developer experience move over quarters, because each depends on governance, review capacity and a body of code that already exists.

The J-curve and the plateau reconcile the same way. One describes a trough at organisational scale, where verification and pipeline work absorb the early gains before the return arrives. The other describes individual savings falling away after an individual peak. Both can be true simultaneously because they are measured at different altitudes and over different clocks — an organisation two quarters into a rollout and a developer two quarters past their own peak are at different points on unrelated curves.

The perceived-against-measured gap is the harder one, and Atlassian's 89-to-6 pair and METR's slower-but-felt-faster result put it squarely on the table. The most defensible reading is that this is first a measurement failure. Executives who cannot produce organisation-wide return figures may still be getting return; developers who felt faster while being slower were measuring against an intuition rather than a clock. A measurement failure and a value failure are different findings, and the evidence currently supports only the first. What follows from that

is that organisations without a verification and measurement layer cannot tell which one they have.

Read that way, the six figures describe a sequence. Tools arrived at the speed at which a developer can install one. Autonomy, assurance and experience move at the speed at which a team can change how it verifies work. What that looks like when one organisation publishes its own before-and-after numbers is the subject of the case study.

Sources referenced in shorthand above are introduced in full in Sources 1–5: [JetBrains](#), [Deloitte](#), [GitKraken](#), [DX](#), [Bessemer](#), [Veracode](#), [METR](#), [Atlassian](#).

8 OF 9

CASE STUDY

Monzo's Agent Chip (2 min)

ONE BANK'S AGENT PLATFORM, AS PUBLISHED 13 AUGUST 2026

Roughly three weeks — from decision to a working MVP, built in-house rather than bought, to avoid vendor lock-in.

Integrations — the agent is reachable from Slack, Linear, GitHub and engineers' local machines.

Architecture — agents run in sandboxed containers on remote infrastructure, behind an MCP gateway.

~10% — of all merged pull requests at Monzo now come from the agent.

1,800 tasks a day — the current running volume.

Nearly all engineers — now use at least one coding agent regularly.

The bottleneck moved — from engineer capacity to reviewer availability.

Monzo published an account of Agent Chip, its internal coding-agent platform, on 13 August 2026 — two days before this issue's window opens. Two of its engineers, Fabien Deshayes and Oli Haley, wrote it. The numbers in it are Monzo's own measurement, self-reported and unaudited, and they are the closest thing in this issue to a like-for-like reference point: a UK bank, publishing its own figures, with a named bottleneck at the end.

The build decision was to make rather than buy, on the grounds that a bought platform would have locked the bank into one vendor's agent at exactly the moment when tool shares are moving by tens of points a year. The MVP took roughly three weeks. Agents reach internal

knowledge sources and tools through an MCP gateway — a single controlled entry point through which every such request passes, rather than each agent holding its own connections. The guardrails around it are architectural: sandboxed container isolation, an isolated proxy for external communication, per-task access configuration, environment-specific images, and secrets kept out of the agent's reach.

That guardrail list is the part of the account that travels furthest. A bank running agents against production repositories has to be able to answer for what the agent could reach, and Monzo's answer is architectural before it is procedural: the agent's reach is bounded by the container, the proxy and the per-task access configuration, ahead of any policy about how engineers should use it.

On the spectrum this issue has been using, Agent Chip is squarely agent-mode. The human review gate was designed in and never removed: engineers still review and merge everything the agent produces. That is the decision that produced the finding.

“An engineer still reviews and merges, but the bottleneck moves from ‘find an engineer with capacity’ to ‘find a reviewer’. That changes who gets to improve the product.”

— Fabien Deshayes and Oli Haley, Monzo, 13 August 2026

Two smaller findings travel with it. Non-engineers at Monzo can now ship small user-experience fixes without waiting to find an available engineer, which is the second half of that quote in practice. And the agent has caught errors during incident response, which puts it inside the operational loop rather than alongside it.

One bank's numbers, one bank's bottleneck, published in full — and a platform that reached a working state in about three weeks, which says as much about how far the underlying tooling has matured as it does about Monzo. What follows from that across the rest of the evidence is the subject of the closing section.

Source: [Monzo](#), [Building Agent Chip](#) (13 August 2026)

9

OF 9

LESSONS & IMPLICATIONS

What the year's evidence means for a team mid-rollout (3 min)

THE VERDICT

Adoption is finished as a question, and the rollout budget is still being written as though it were the only one. The measurable effect of near-universal agent use is that the same amount of judgement now has to be applied to several times as much output, by the same number of people, in the same week. The teams whose numbers hold up are the ones that treated the scaffolding around the agent — the instruction file, the test command, the review gate, the attribution layer — as the work, rather than as preparation for it.

Review capacity is the variable that binds

Every downstream measure in this issue points at the same constrained resource. Pull requests are larger, merge volume is higher, cross-agent conflicts arrive structurally rather than textually, and the security pass rate means a substantial share of generated code needs a human security judgement. Osmani's point about reconstruction explains why the minutes do not divide the way they used to, and Monzo found the same constraint by removing the one in front of it. The countermeasures the accounts converge on are small and procedural: a size limit on agent pull requests, an intake standard that carries the reasoning and the test evidence, and review tiered by the risk of the code path rather than by who wrote it.

The bands belong in any internal target

A 90% adoption target and a 28% autonomous-working target are two entirely different programmes, with different governance, different risk and different costs. The vocabulary problem in the published research is also a vocabulary problem inside organisations, and it is cheaper to fix in a target than in a post-mortem. The same precision applies to the norm a team believes it holds: 79% of agent pull requests in the LeadDev analysis had the same person reviewing and modifying the work, which is what a stated review norm looks like when nothing enforces it.

Verification is a standing cost line

DORA models the verification tax as part of a dip that a rollout climbs out of. The four years of GitClear data and the flat security pass rate describe something that has not improved with model capability. A cost line that has held steady across four testing snapshots and four years of commit history is a permanent one, and budgeting it as a transition expense understates it. Two of Bessemer's four named blockers — evaluating code quality and measuring productivity gains — describe the same missing layer, and Figma's per-pull-request review cost shows what filling it actually prices at.

KEY TAKEAWAY

- Review capacity is the variable a plan binds on, and diff size is the lever teams reach for first.

- The spectrum bands belong in any internal target: “90% adoption” and “28% autonomous” are not the same commitment.
- The stated review norm and the observed one diverge quietly; the LeadDev analysis is what the divergence looks like measured.
- A runnable test command and a short instruction file are the cheapest scaffolding in the pack, and every practitioner account starts there.
- The security review layer is a permanent cost line, not a transitional one — the 56% pass rate has not moved in a year.
- Removing every access barrier does not produce adoption on its own; Sentry removed all three it could name and still reports a plateau.
- Visible peer use is what spreads a rollout, and per-user cost attribution is cheaper to build before the bill than after it.
- The junior-pipeline decision made this year sets the supervision capacity available in three.

Sources: the studies, teams and rollouts cited in Sources 1–5, the Counterpoint and the case study above — among them [DORA](#), [GitClear](#), [Addy Osmani](#), [Sentry](#) and [Monzo](#).

MEANWHILE ...

The Rest of the Week

While this issue was deep in the agent literature, the week kept moving.

WOULD HAVE LED

Two days of UK retail-banking degradation, and two different failure responses. On 18 August a technical fault locked Lloyds, Halifax and Bank of Scotland customers out of mobile and online banking; Downdetector logged more than 2,450 reports for Lloyds, more than 1,000 for Halifax and several hundred for Bank of Scotland, around 70% of them app-related, with problems spiking by 11:30am and service restored shortly after 1:00pm. On 19 August more than 3,000 Monzo customers reported problems after midday, and Monzo moved into its published stand-in system — a fallback mode in which core payment functions keep running while other services are unavailable. Monzo’s statement: “We’ve moved into our back-up system Monzo Stand-in and customers can still do most things on their Monzo app including sending bank transfers and making card payments and ATM withdrawals.” Neither bank has published a cause for either incident. [IBTimes UK](#) · [Evening Standard via AOL](#)

WOULD HAVE LED

Visa, Mastercard and around two dozen others form an Agentic Payments Alliance — and name loyalty as unsolved. Rain launched the alliance on 18 August with about two dozen founding members including Visa, Mastercard, Fiserv, Circle, Remitly, Shift4, Sardine, Lithic, Evertec, Episode

Six and Chainalysis, to standardise agent identification and authorisation, run shared research and shape regulatory advocacy. American Banker reports that much of the infrastructure guiding agentic AI — how agents get authorised, how fraud gets caught, and how loyalty and rewards work — is still being defined. “The risk in a moment like this is not that the industry moves too slowly — it’s that innovation outpaces alignment,” said Sherri Haymond, executive vice president and global head of digital commercialization at Mastercard. [American Banker](#)

Apple patches an image-processing flaw reported by Meta’s red team. Security updates for recent iPhones, iPads, Macs and Vision Pro address an integer-overflow bug in ImageIO, the framework Apple uses to decode images, which could allow code execution when a malicious image is processed; Apple fixed it with improved input validation. [The Register](#), 18 August 2026

Meta ships Muse Code, a terminal coding agent for large codebases, in beta. Announced on 5 August, before this window opens, it fans large jobs out to parallel sub-agents working in isolated worktrees, and Meta positions it on cost against Codex and Claude Code. No pricing figures were published. [TechCrunch](#), 5 August 2026

ON THE RADAR

Coming Up

4 Aug **keyv and cacheable npm packages hijacked** — Socket reports at least ten packages in the keyv and cacheable namespaces republished with a credential-sweeping preinstall hook, collectively accounting for tens of millions of weekly downloads, after a maintainer account was compromised and stolen tokens were used to republish further packages. The leadership question is which build agents hold tokens. [Socket](#)

27 Jul **Dynatrace autonomous SRE agent and no-code Agent Builder** — announced with availability stated as August 2026; no general-availability date has been published, so this remains an announcement rather than an event. [Dynatrace](#)

2 Dec 27 **EU AI Act high-risk regime slips** — Annex III stand-alone high-risk obligations covering hiring, credit scoring, education and critical infrastructure are deferred to 2 December 2027, and Annex I embedded high-risk AI to 2 August 2028. Article 50 transparency obligations were not deferred and took effect on 2 August 2026. [Gibson Dunn](#)

RECOMMENDED READING

Sources & Further Reading

One vendor's telemetry, attributed as such: "Claude Code sessions run on the most capable models far more often (54% are served by Opus, against 10% of chat and Cowork conversations)", with sessions increasingly consisting of long-running agentic tasks.

Generative AI as Seniority-Biased Technological Change

Hosseini Maasoum & Lichtinger, SSRN

The working paper behind Source 4's direction-only finding, for the authors' own magnitudes. The SSRN page was not reachable during preparation of this issue.

Engineering Progression When AI Ate the Middle

Ben Linders, InfoQ

Alasdair Allan's QCon London talk, connecting this year's hiring decision to the supervision capacity available in three. Recommended rather than cited, because its figures are a secondary aggregation.

Inside AI-pilled engineering teams

Bessemer Atlas

The full account of the enablement layer summarised in Source 5, including the Ramp and Shopify practices.

ROI of AI-assisted Software Development (v.2026.1)

DORA

The model behind the finance conversation in Source 3, published with an accompanying ROI calculator.

Engineering practices that make coding agents work

Simon Willison

The transcript behind Source 2's testing material, and the densest single hour of practice in the pack — repo readiness, session conventions and where agent-written code still needs a refinement pass.

Harness engineering for coding agent users

Birgitta Böckeler, martinowler.com

The guides-and-sensors split in full, with the argument for treating the scaffolding around the agent as a staffed, maintained artefact.

Agentic Code Review

Addy Osmani

The pull-request intake standard and risk-tiered review policy summarised in Source 2, with the tool-overlap data behind them.

VOL. II

Engineering Digest

21 August 2026 · Deep Dive · Issue #23

Next edition: 28 August 2026

ARCHIVE

#18 · Deep Dive

#19 · Weekly

#20 · Weekly

#21 · Weekly

#22 · Weekly

#23 · Deep Dive